Chemical Data Extractor

Export Formats Reference

What each downloaded file contains, and which tool opens it

Version 3.2 · Data dictionary
Column-level detail for nodes.csv, edges.csv, the GraphML, Cypher, RDF, Parquet and JSON exports, plus guidance on choosing the right file for the job.

Generated from the application's documentation source (`docs/tutorial/export-formats.md`), so it always matches the shipped interface.

Contents

Export Formats Reference 3

1. nodes.csv — the entity table 3
2. edges.csv — the relationship table 3
3. _Knowledge_Graph.xlsx — the workbook 3
4. graph.graphml — the network 3
5. neo4j_import.cypher — for Neo4j 4
6. graph.ttl — RDF / linked data 4
7. nodes.parquet and edges.parquet — large graphs 4
8. JSON exports 4
9. enriched_data.xlsx and structure_images/ 4
10. Choosing quickly 4

Export Formats Reference

Chemical Data Extractor v3.2 · what each downloaded file contains

Every completed search writes up to 15 files. This reference lists what each one holds and the tool to open it with. Files are removed from the server about 30 minutes after the search completes.

1. `nodes.csv` — the entity table

One row per resolved entity. This is the file to start from.

Column group	Typical columns	Notes
Identity	`uid:ID`, `name`, `:LABEL`	`uid:ID` is the primary key used by `edges.csv`
Type	`:LABEL`	`COMPOUND`, `PROTEIN`, `STRUCTURE`, `ENZYME` and others
Chemistry	`smiles`, `formula`, `molecular_weight`	Enriched from PubChem and ChEMBL after extraction
Bioactivity	`bioactivity_summary`, `pchembl_value`, `max_phase`	`pchembl_value` is the potency ranking value; higher is stronger
Provenance	cross-references, evidence, synonyms	Which database each fact came from

Empty cells mean the source database had no value — they are not errors.

2. `edges.csv` — the relationship table

One row per relationship.

Column	Meaning
`:START_ID`	Source node `uid:ID` — join to `nodes.csv`
`:END_ID`	Target node `uid:ID` — join to `nodes.csv`
`:TYPE`	Relationship type: `INHIBITS`, `BINDS`, `HAS_STRUCTURE`, …
`activity_type`	Assay type, e.g. `IC50`, `Ki`
`activity_value`	The measured value
`units`	Units for that value
`pchembl_value`	Normalised potency for the activity

Joining `edges.csv` to `nodes.csv` on `uid:ID` gives a complete, analysis-ready table pair.

3. `<query>_Knowledge_Graph.xlsx` — the workbook

The multi-sheet workbook. It is the most convenient file for most people.

Sheet	Contents
Summary	Entity counts, relationship counts, ranking summary
Nodes	The same data as `nodes.csv`, formatted for reading
Edges	The same data as `edges.csv`
Bioactivity	Potency values with pChEMBL ranking

4. `graph.graphml` — the network

The complete graph in GraphML, the standard interchange format for network tools.

- **Gephi** — open the file directly; nodes and edges are typed automatically.
- **Cytoscape** — import as a network; relations appear as edge attributes.

- **yEd** — import as a graph; edge labels show the relation type.

Use this file whenever the shape of the network is the result you need.

5. `neo4j_import.cypher` — for Neo4j

A script of `CREATE` statements that rebuilds the graph in Neo4j.

```
cypher-shell -u neo4j -p <password> < neo4j_import.cypher
```

Run it against an empty database. Re-running it duplicates every node, so clear the database first or use a labelled import.

6. `graph.ttl` — RDF / linked data

Turtle syntax: triples of subject, predicate and object.

- **Blazegraph** — load into a named graph.
- **Jena / Fuseki** — import into a model.
- **rdflib (Python)** — `g = rdflib.Graph().parse("graph.ttl")`

7. `nodes.parquet` and `edges.parquet` — large graphs

Columnar versions of the two CSV tables. They are much smaller and faster to query than CSV once a run produces tens of thousands of rows.

```
import pandas as pd
nodes = pd.read_parquet("nodes.parquet")
edges = pd.read_parquet("edges.parquet")
```

DuckDB, Spark and Arrow all read Parquet natively.

8. JSON exports

File	Contains	Use for
`rdf_graph.json`	RDF-ready dictionaries	Post-processing in Python
`triples.json`	Flat subject–predicate–object triples	Custom graph stores
`vector_index_metadata.json`	Metadata for an embedding index	Retrieval and RAG pipelines
`search_library.json`	Resolved query terms and routing decisions	Reproducing a search exactly

9. `enriched_data.xlsx` and `structure_images/`

Produced by the enrichment stage near the end of a run.

- `enriched_data.xlsx` — compounds with SMILES, molecular properties and publication metadata added after extraction.
- `structure_images/` — one rendered 2D structure image per compound, for figures and visual screening.

If a run was started with enrichment disabled, these are absent; that is why a file list can be shorter than this reference.

10. Choosing quickly

If you want to…	Download
Look at the data in a spreadsheet	`*_Knowledge_Graph.xlsx`
Analyse in pandas or R	`nodes.csv`, `edges.csv`
Draw the network	`graph.graphml`

Load a graph database	`neo4j_import.cypher`
Work with linked data	`graph.ttl`
Handle a very large result set	`nodes.parquet`, `edges.parquet`
Make a figure for a paper	`enriched_data.xlsx`, `structure_images/`

Retention: downloads are deleted from the server roughly 30 minutes after the search finishes. Save anything you need immediately.